

Methods of the TwoLaneTrack

Methods of the TwoLaneTrack

- The *methods* listed in the table of contents to the left.
- The **[_Methods of Curved Objects](#)**.
- The **[Methods of the Material Flow Objects](#)**.
- The **[Methods of All Objects](#)**.

A number of methods, which the *TwoLaneTrack* shares with the other material flow objects, apply to a lane, instead of to the entire object.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



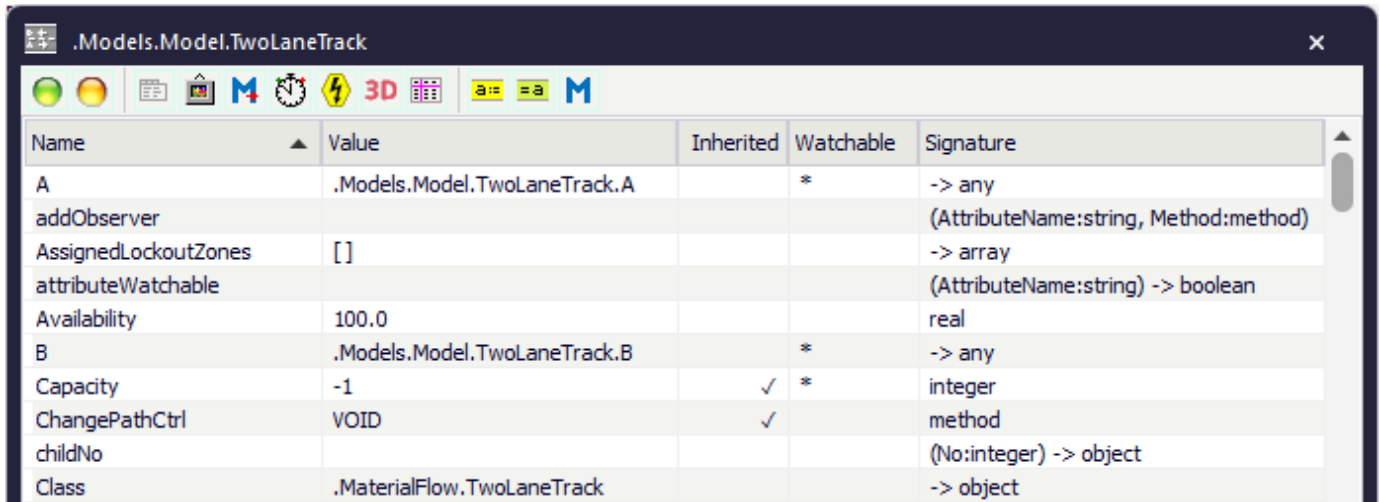
The screenshot shows the .Models.Model.Station window. The table below is a representation of the content shown in the image.

Name	Value	Inherited	Watchable	Signature
moveToFolder				(Folder:object) -> object
MU	VOID			([Index:integer:=1]) -> object
MUPart	VOID			([Index:integer:=1]) -> object

Below the table, the following labels are provided for the columns:

- Name of the method
- Identifier and data type of the parameter you enter
- Default value
- Data type of the return value

For the *TwoLaneTrack* the dialog shows **A** and **B** for the respective lane, but not the methods proper for these lanes.



Name	Value	Inherited	Watchable	Signature
A	.Models.Model.TwoLaneTrack.A		*	-> any
addObserver				(AttributeName:string, Method:method)
AssignedLockoutZones	[]			-> array
attributeWatchable				(AttributeName:string) -> boolean
Availability	100.0			real
B	.Models.Model.TwoLaneTrack.B		*	-> any
Capacity	-1	✓	*	integer
ChangePathCtrl	VOID	✓		method
childNo				(No:integer) -> object
Class	.MaterialFlow.TwoLaneTrack			-> object

You can:

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *attributes* and *methods* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *attributes* and *methods* of the selected [Instance \[general description\]](#).

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.A.exitBlockList([ExitBlockingList:table]) -> object[]  
<Path>.B.exitBlockList([ExitBlockingList:table]) -> object[]
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
The expression A or B designates the lane of the *TwoLaneTrack* that you want to access.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

Note

Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

If a parameter has a default value, the signature shows the default value after the parameter
If the *method* has a return value, the signature shows its data type after the arrow ->.

bwBlockList [SimTalk] - lane A or B

Returns the blocking list in the backward direction of the specified lane of the *TwoLaneTrack* designated by <Path>.

Remarks

Specify the optional parameter of data type *table*, to make *bwBlockList* write the objects contained in the **Backward Blocking List** into this list.

Do not specify the parameter, to make *bwBlockList* return an *array* containing the objects contained in the **Backward Blocking List**. The *array* is one-dimensional and only contains the objects in the blocking list. If you need the start times of the blockages, which are contained in the blocking table, you can query them from the object with the attribute *BlockingStartTime*.

Type

Method

Syntax

```
<Path>.A.bwBlockList([BackwardBlockingList:table]) → any
<Path>.B.bwBlockList([BackwardBlockingList:table]) → any
```

Parameter

The optional parameter *BackwardBlockingList* of data type *table* designates the name of the table, which can be a *DataTable* or a local variable, into which the method writes the values.

Plant Simulation automatically generates the format of this table with two columns.

Column 1 has the data type *object* and contains all *Transporters* that unsuccessfully tried to move to the object.

Column 2 contains the simulation time at which the *Transporters* attempted to enter the object. The number of rows matches the number of *Transporters* in the blocking list.

Return Value

The return value has the data type *any*.

Is an *array* containing all *MUs* in the **Backward Blocking List** of the *TwoLaneTrack* on the specified lane if you do not specify the optional parameter.

Example

```
MyTwoLaneTrack.B.bwBlocklist(DataTable)
```

SimTalk

[BlockingStarttime \[SimTalk\]](#)

[BwBlockListEntry1 \[SimTalk\]](#)

See also

[Backward Blocking List](#)

[Check the Contents List of the Stations](#)

contentsList [SimTalk] - lane A or B

Returns the entire **Contents**, i.e., all *Transporters* which are located on the specified lane of the **Contents List** of *TwoLaneTrack* designated by <Path>.

Remarks

Specify the optional parameter of data type *table* to write the objects contained in the **Contents List** into this list.

Do not specify the parameter to return an *array* containing the objects contained in the **Contents List**.

Type

Method

Syntax

```
<Path>.A.contentsList([ContentsList:table]) → any
<Path>.B.contentsList([ContentsList:table]) → any
```

Parameter

The optional parameter *ContentsList* of data type *table* designates the name of the table. *Plant Simulation* automatically generates the format of this table and deletes any existing format and contents of the table. The contents list is a table with three columns. The data type of column 1 is *object*, of the second and third *length*. Column 1 contains the path to the *Transporters*, column 2 the start position of the *MU* part, and column 3 the end position in the length unit you specified. The number of rows matches the number of *Transporter* parts located on the object.

Return Value

The return value has the data type *any*.

Is an *array* containing the objects in the **Contents List** if you do not specify the optional parameter.

Examples

```
MyTwoLaneTrack.A.contentsList(DataTable)
```

```
print TwoLaneTrack.A.contentsList
// might, for example, return [.MUs.Transporter:314, 34.488399, 35.988399]
[.MUs.Transporter:316, 32.988399, 34.488399][.MUs.Transporter:318,
```

```
31.488399, 32.988399][.MUs.Transporter:320, 29.988399, 31.488399] in the Console
```

```
print TwoLaneTrack.A.contentsList  
// might, for example, return [.MUs.Transporter:314, 34.488399, 35.988399]  
[.MUs.Transporter:316, 32.988399, 34.488399][.MUs.Transporter:318,  
31.488399, 32.988399][.MUs.Transporter:320, 29.988399, 31.488399] in the Console
```

SimTalk

[contentsList \[SimTalk\] - material flow objects](#)

See also

[Contents \[material flow objects\]](#)

[Check the Contents List of the Stations](#)

exitBlockList [SimTalk] - lane A or B

Returns the **Exit Blocking List** of the specified lane of the *TwoLaneTrack* designated by <Path>.

Remarks

Specify the optional parameter *ExitBlockingList* of data type *table* to write the objects contained in the **Backward Blocking List** into this list.

Do not specify the parameter *ExitBlockingList* to return an *array* containing the objects contained in the **Backward Blocking List**.

The *array* is one-dimensional and only contains the objects in the blocking list. If you need the start times of the blockages, which are contained in the blocking table, you can query them from the object with the attribute *BlockingStarttime*.

Type

Method

Syntax

```
<Path>.A.exitBlockList([ExitBlockingList:table]) -> object[]
<Path>.B.exitBlockList([ExitBlockingList:table]) -> object[]
```

Parameter

The optional parameter *ExitBlockingList* of data type *table* designates the name of the table, which can be a *DataTable* or a local variable, into which the method writes the values.

Plant Simulation automatically generates the format of this table with two columns.

Column 1 has the data type *object* and contains the *Transporters* that unsuccessfully tried to exit the object.

Column 2 contains the simulation time at which the *Transporters* attempted to enter the object.

Return Value

The return value has the data type *any*.

Example

```
MyTwoLaneTrack.B.exitBlockList(DataTable)
```


SimTalk

[BlockingStarttime \[SimTalk\]](#)

See also

[Exit Blocking List](#)

[Check the Contents List of the Stations](#)

fwBlockList [SimTalk] - lane A or B

Returns the **Blocking List** in the forward direction of the specified lane of the *TwoLaneTrack* designated by <Path>.

Remarks

Specify the optional parameter of data type *table* to write the objects contained in the **Forward Blocking List** into this list.

Do not specify the parameter to return an *array* containing the objects contained in the **Forward Blocking List**.

The *array* is one-dimensional and only contains the objects in the blocking list. If you need the start times of the blockages, which are contained in the blocking table, you can query them from the object with the attribute *BlockingStarttime*.

Type

Method

Syntax

```
<Path>.A.fwBlockList([ForwardBlockingList:table]) -> any  
<Path>.B.fwBlockList([ForwardBlockingList:table]) -> any
```

Parameter

The optional parameter *ForwardBlockingList* of data type *table* designates the name of the table, which can be a *DataTable* or a local variable, into which the method writes the values.

Plant Simulation automatically generates the format of this table with two columns.

Column 1 has the data type *object* and contains the *Transporters* that unsuccessfully tried to move to the object.

Column 2 contains the simulation time at which the *Transporters* attempted to enter the object. The number of rows matches the number of *Transporters* in the blocking list.

Return Value

The return value has the data type *any*.

Example

```
MyTwoLaneTrack.B.fwBlockList(DataTable)
```

SimTalk

[BlockingStarttime \[SimTalk\]](#)

See also

[Forward Blocking List](#)

[Check the Contents List of the Stations](#)

getRouteLength [SimTalk] - lane A or B

Returns the shortest route from the lane of the *TwoLaneTrack* designated by <Path> to the target and returns the length of this route.

Remarks

The destination can either be a material flow object, which can be reached directly via *Connectors*, or a destination, which you typed into a sensor.

Note

As the route is intended for *Transporters*, *Plant Simulation* only takes objects of type *Track* and *TwoLaneTrack* into consideration, when computing the route.

Type

Method

Syntax

```
<Path>.A.getRouteLength(Target:path[, Backwards:boolean, Position:length,
ObjectsAlongRoute:table, RouteWeightingAttribute:string] → length
<Path>.B.getRouteLength(Target:path[, Backwards:boolean, Position:length,
ObjectsAlongRoute:table, RouteWeightingAttribute:string] → length
```

Parameters

You can specify the following parameters:

- The parameter *Target* of data type *object* designates the target.
- The optional parameter *Backwards* of data type *boolean* designates the direction in which *Plant Simulation* looks for a route. *True* stands for backward, *false* stands for forward, which is the default setting.
- The optional parameter *Position* of data type *length* designates the position on the lane at which the search starts. If you do not specify the parameter, *Plant Simulation* starts searching at the end of the *lane* if you search forward, or it starts searching at the beginning if you search backward. This is particularly important if the **Destination** is a sensor, which is located on the lane of the *TwoLaneTrack* itself.
- The optional parameter *ObjectsAlongRoute* of data type *table* writes the objects along the route into the specified table.
- The optional parameter *RouteWeightingAttribute* of data type *string* sets the name of the attribute for the route weighting attribute for automatic routing. If the parameter is not passed, *Plant Simulation* does not weight the lengths of the routes.

Return Value

The return value has the data type *length*.

Is -1 if *Plant Simulation* did not find a route.

Example

```
print TwoLaneTrack5.B.getRouteLength(Track26)
```

SimTalk

[Destination \[SimTalk\] - sensors](#)

[RouteWeightingAttr \[SimTalk\] - Transporter](#)

pred [SimTalk] - lane A or B

Returns the direct predecessor of the designated lane of the *TwoLaneTrack* designated by <Path>.

Type

Method

Syntax

```
<Path>.A.pred([PredecessorNumber:integer:=1]) → object  
<Path>.B.pred([PredecessorNumber:integer:=1]) → object
```

Parameter

The optional parameter *PredecessorNumber* of data type *integer* designates the n-th predecessor of the specified lane of the *TwoLaneTrack*. The objects have to be connected for this to work.

Default Value of the Parameter

The default value is 1.

Return Value

The return value has the data type *object*.

Example

```
if MyTwoLaneTrack.A.pred(3) = TwoLaneTrack1  
  ...  
end
```

See also

[succ \[SimTalk\] - lane A or B](#)

predConnector [SimTalk] - lane A or B

Returns the incoming *Connector* connected directly with the specified lane of the *TwoLaneTrack* designated by <Path>. Adding and deleting connections may change the index number of a connection.

Type**Method****Syntax**

```
<Path>.A.predConnector([PredecessorNumber:integer:=1]) → object
<Path>.B.predConnector([PredecessorNumber:integer:=1]) → object
```

Parameter

The optional parameter *PredecessorNumber* of data type *integer* designates the n-th preceding *Connector* of the specified lane of the *TwoLaneTrack*.

Default Value of the Parameter

The default value is 1.

Return Value

The return value has the data type *object*.

VOID if the connection does not exist.

Example

```
for var i := MyTwoLaneTrack.B.NumPred downto 1 // deletes connections
  MyTwoLaneTrack.B.predConnector(i).deleteObject
next
```

SimTalk**succConnector [SimTalk] - lane A or B**

predLane [SimTalk] - lane A or B

Returns the direct predecessor of the designated lane of the *TwoLaneTrack* designated by <Path>.

Type

Method

Syntax

```
<Path>.A.predLane([Predecessor:integer]) → any  
<Path>.B.predLane([Predecessor:integer]) → any
```

Parameter

The optional parameter *Predecessor* of data type *integer* designates the lane of the n-th predecessor of the *TwoLaneTrack*. The objects have to be connected for this to work.

Return Value

The return value has the data type *any*.

Example

```
if MyTwoLaneTrack.A.predLane(3) = TwoLaneTrack1.B  
  ...  
end
```

SimTalk

succLane [SimTalk] - lane A or B

predLaneNo [SimTalk] - lane A or B

Returns the number of the preceding lane of the specified lane of the *TwoLaneTrack* designated by <Path>.

Remarks

Only works if the objects are connected.

Type

Method

Syntax

```
<Path>.A.predLaneNo([Predecessor:integer]) → integer
<Path>.B.predLaneNo([Predecessor:integer]) → integer
```

Parameter

The optional parameter *Predecessor* of data type *integer* designates the number of the lane of the n-th predecessor of the *TwoLaneTrack*. The objects have to be connected for this to work.

Return Value

The return value has the data type *integer*.

Example

```
if MyTwoLaneTrack.A.predLaneNo(3) = 2
  print "third preceeding lane is lane B"
end
```

SimTalk**succLaneNo [SimTalk] - lane A or B**

succ [SimTalk] - lane A or B

Returns the direct successor of the specified lane of the *TwoLaneTrack* designated by <Path>.

Type

Method

Syntax

```
<Path>.A.succ([SuccessorNumber:integer:=1]) → object
<Path>.B.succ([SuccessorNumber:integer:=1]) → object
```

Parameter

The optional parameter *SuccessorNumber* of data type *integer* designates the number of the successor on the specified lane of the *TwoLaneTrack*. The objects have to be connected for this to work.

Default Value of the Parameter

The default value is 1, meaning the first successor on the lane.

Return Value

The return value has the data type *object*.

Note

If the n-th successor of the object is an *Interface*, *Plant Simulation* does not return the *Interface* itself, but the successor of the *Interface*. If this again is an *Interface*, it will be tracked until *Plant Simulation* reaches a material flow object which can accept *MUs*.

If the n-th successor of the object is a *Frame*, *Plant Simulation* does not return the *Frame* itself, but the successor of the *Interface* inserted into this *Frame*.

If an *Interface* only has a single successor, this successor is uniquely identified and will be used, independent of the exit strategy of the *Interface*.

If an *Interface* has several successors, the successor is identified according to its exit strategy. If you selected a blocking exit strategy, the strategy determines the successor no matter if it can accept a *MU* at the moment or not.

If you selected a non-blocking exit strategy, *Plant Simulation* searches for the next successor that can accept a *MU* according to the selected exit strategy.

If none of the successors can accept a *MU*, the method returns VOID. The successor that can accept a *MU* next cannot be determined at that point in time. This does not apply if there is only a single successor.

Example

```
if MyTwoLaneTrack.B.succ = TwoLaneTrack1
  ...
end
```

SimTalk

pred [SimTalk] - lane A or B

succConnector [SimTalk] - lane A or B

Returns the outgoing *Connector* connected directly with the specified lane of the *TwoLaneTrack* designated by <Path>.

Remarks

Adding and deleting connections may change the index number of a connection.

Type

Method

Syntax

```
<Path>.A.succConnector([SuccessorNumber:integer:=1]) → object  
<Path>.B.succConnector([SuccessorNumber:integer:=1]) → object
```

Parameter

The optional parameter *SuccessorNumber* of data type *integer* designates the *successor* on the specified lane of the *TwoLaneTrack*. The objects have to be connected for this to work.

Default Value of the Parameter

The default value is 1, meaning the first succeeding connector on the lane.

Return Value

The return value has the data type *object*.

VOID if the connection does not exist.

Example

```
myTwoLaneTrack.B.succConnector.deleteObject
```

SimTalk

predConnector [SimTalk] - lane A or B

succLane [SimTalk] - lane A or B

Returns the direct successor of the designated lane of the *TwoLaneTrack* designated by <Path>.

Type

Method

Syntax

```
<Path>.A.succLane([Successor:integer]) → any
<Path>.B.succLane([Successor:integer]) → any
```

Parameter

The optional parameter *Successor* of data type *integer* designates the lane of the n-th successor of the *TwoLaneTrack*. The objects have to be connected for this to work.

Return Value

The return value has the data type *any*.

Example

```
if MyTwoLaneTrack.B.succLane = TwoLaneTrack1.A
  ...
end
```

SimTalk**predLane [SimTalk] - lane A or B**

succLaneNo [SimTalk] - lane A or B

Returns the number of the succeeding lane of the specified lane of the *TwoLaneTrack* designated by <Path>.

Type

Method

Syntax

```
<Path>.A.succLaneNo([LaneNumber:integer]) → integer  
<Path>.B.succLaneNo([LaneNumber:integer]) → integer
```

Parameter

The optional parameter *LaneNumber* of data type *integer* designates the number of the lane of the n-th successor lane of the *TwoLaneTrack*. The objects have to be connected for this to work.

Return Value

The return value has the data type *integer*.

Example

```
if MyTwoLaneTrack.A.succLaneNo = 1  
  print "succeeding lane is lane A"  
end
```

SimTalk

predLaneNo [SimTalk] - lane A or B

Read-Only Attributes of the TwoLaneTrack

_Read-Only Attributes of the TwoLaneTrack

The *TwoLaneTrack* provides:

- The *read-only attributes* listed in the table of contents to the left.
- The [_Read-Only Attributes of All Objects](#).
- The [Read-Only Attributes of the Material Flow Objects](#).